XLA Face Privacy System

Hệ Thống Camera Bảo Vệ Quyền Riêng Tư Khuôn Mặt Theo Thời Gian Thực

Thành viên nhóm

Vai trò	Tên	Phụ trách
Leader	**Đạt**	Security + Blur
Backend	Khánh	FastAPI + Pipeline
Detection	Khoa	YOLO + FaceDB

Thông tin

- Môn: Xử Lý Ảnh
- Ngôn ngữ: Python 3.13
- Framework: FastAPI
- Model: YOLOv11n-face

Vấn Đề: Camera An Ninh vs Quyền Riêng Tư

Thực trạng

- Camera an ninh quay **tất cả mọi người**
- Khách, người giúp việc bị nhận dạng không đồng ý
- Vi phạm GDPR, quyền riêng tư cá nhân
- Dữ liệu biometric lưu plaintext = nguy cơ rò rỉ

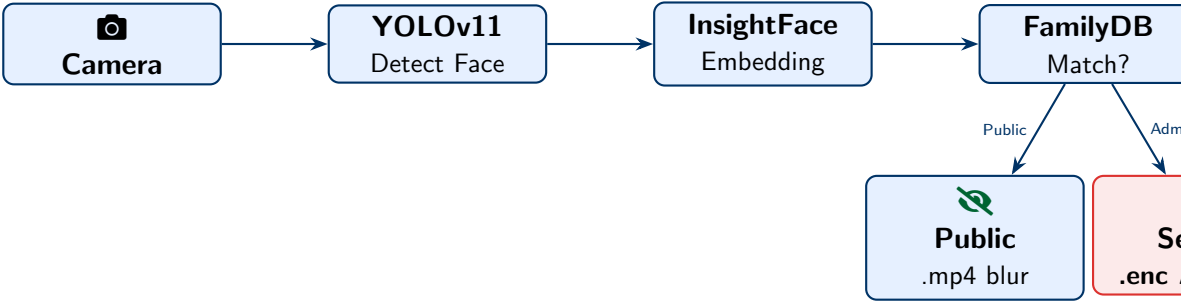
Hai thái cực

Ghi thô	**Xóa hết**
Lộ danh tính	Mất bằng chứng
Vi phạm privacy	Không điều tra được
✗ Privacy	✗ Evidence

Câu hỏi đặt ra

Có thể **bảo vệ quyền riêng tư** mà **không mất bằng chứng** không?

Giải Pháp: Privacy by Default, Evidence on Demand



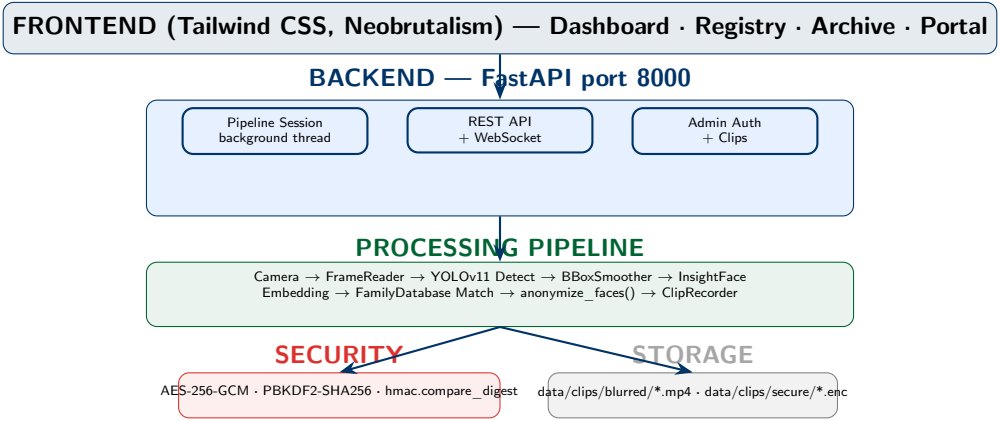
Tầng 1: Blurred Stream

- Video đã che mặt người lạ
- Lưu định dạng `.mp4` thường
- Mọi người trong hệ thống xem được

Tầng 2: Encrypted Store

- Video gốc, khuôn mặt nguyên vẹn
- Mã hóa **AES-256-GCM**
- *Chỉ* Admin biết password mới xem được

Kiến Trúc Tổng Thể



Cấu Trúc Mã Nguồn

Phụ trách: Khoa

`src/detector.py`	YOLOv11 wrap
`src/face_recognition.py`	InsightFace 512-
`src/family_database.py`	Face DB CRUD
`src/bbox_smoother.py`	EMA smoothing
`models/yolov11n-face.pt`	YOLO weights

Phụ trách: Khánh

`src/app_api.py`	FastAPI 1700+ dòn
`src/backend_api.py`	Pipeline runner
`src/run_phone_cam.py`	FrameReader + Tra
`frontend/*.html`	6 trang HTML

Phụ trách: Đạt (Leader)

`src/privacy.py`	7 blur mode
`src/anonymizer_backend.py`	Config + Pres
`src/clip_recorder.py`	AES-256-GCM
`src/admin_auth.py`	PBKDF2 + ve
`src/face_lock.py`	Per-face AES

Data

`data/family_members.json`	Embedding
`data/admin_credentials.json`	Salt + Hash
`data/clips/secure/*.enc`	Encrypted c
`data/clips/blurred/*.mp4`	Public clips

YOLOv11n-face — Phát Hiện Khuôn Mặt

Tại sao YOLOv11n?

- **Nano**: tối ưu inference trên CPU
- Fine-tuned đặc biệt cho face detection
- Hỗ trợ NMS (Non-Max Suppression)
- conf_threshold = **0.35**, iou_threshold = **0.50**

Optimize: detect_every

Frame xen kẽ: chỉ gọi YOLO mỗi **2 frame**.
Frame bỏ qua dùng lại kết quả lần trước.
⇒ Giảm 50% compute, lag không đáng kể.

```
class FaceDetector:
  def detect_faces(self, frame):
    results = self.model(
      frame,
      imgsz=512,          # inference size
      conf=0.35,          # min confidence
      iou=0.50,           # NMS threshold
      verbose=False,
    )
    boxes = []
    for r in results:
      for box in r.boxes:
        x1,y1,x2,y2 = map(int, box.xyxy[0])
```

InsightFace Buffalo-L — Nhận Diện Thành Viên

Embedding 512 chiều

- Model **buffalo_l**: state-of-the-art recognition
- Vector 512 chiều = "vân tay số" khuôn mặt
- Cosine similarity: -1 (hoàn toàn khác) đến $+1$ (giống hệt)
- Threshold: **0.35**
  - $> 0.35 \Rightarrow$ thành viên $\Rightarrow$ **không blur**
  - $\leq 0.35 \Rightarrow$ người lạ $\Rightarrow$ **blur**

Vectorized Matching

So sánh *một lần* với **toàn bộ** embedding matrix:

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \cdot \|b\|}$$

```python
class FaceRecognizer:
  def __init__(self):
    self.app = FaceAnalysis(
      name="buffalo_l",
      providers=[
        "CPUExecutionProvider"
      ]
    )
    self.app.prepare(
      ctx_id=0,
      det_size=(256, 256)
    )

  @staticmethod
  def cosine_similarity(a, b):
    return float(
      np.dot(a, b) / (
        np.linalg.norm(a)
```

Face Registry — Quản Lý Thành Viên

Quy trình đăng ký

1. Admin upload ảnh thành viên (POST /members)
2. Backend extract InsightFace embedding
3. Lưu vector vào `data/family_members.json`
4. Pipeline tự động load lại database

Database hiện tại

Thành viên	Ảnh	Threshold
Đạt	100	0.35
Khoa	100	0.35

Database Schema

```
{
  "member_id": "dat",
  "name": "Dat",
  "threshold": 0.35,
  "embeddings": [
    [0.12, -0.34, ...],  // 512-dim
    [0.11, -0.33, ...],  // mau 2
```

Tại sao 100 ảnh?

Nhiều góc độ + điều kiện ánh sáng khác nhau ⇒ embedding ổn định hơn ⇒ giảm false positive

API

`GET /members` — list

FastAPI Backend — REST + WebSocket

Endpoint Groups

`GET /health`	Server status
`GET /presets`	4 config presets
`POST /anonymize`	Blur 1 ảnh upload
`POST /lock`	AES per-face lock
`POST /recognize`	Nhận diện ảnh tĩnh
`GET /cameras`	List cameras
`POST /pipeline/start`	Start live pipeline
`PATCH /pipeline/config`	Hot-reload config
`GET /stream/mjpeg`	MJPEG stream
`WS /stream/ws`	WebSocket binary
`GET /clips`	List clips
`POST /clips/decrypt`	Admin decrypt

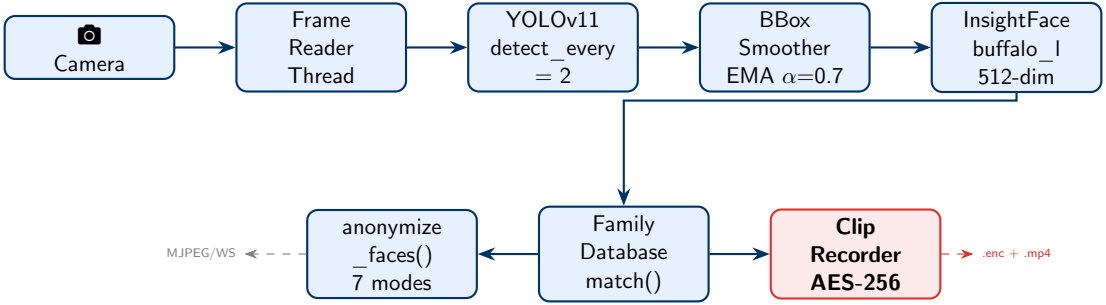
4 Anonymization Presets

Preset	Mode	YOLO
`fast`	pixelate	416
`balanced`	headcloak	512
`strong`	obliterate	512
`strict`	headcloak	416

Hot-reload Config

`PATCH /pipeline/config`
Đổi mode, threshold, proc_width **không cần restart server**.
⇒ Production-ready

Live Pipeline — Luồng Xử Lý Real-time



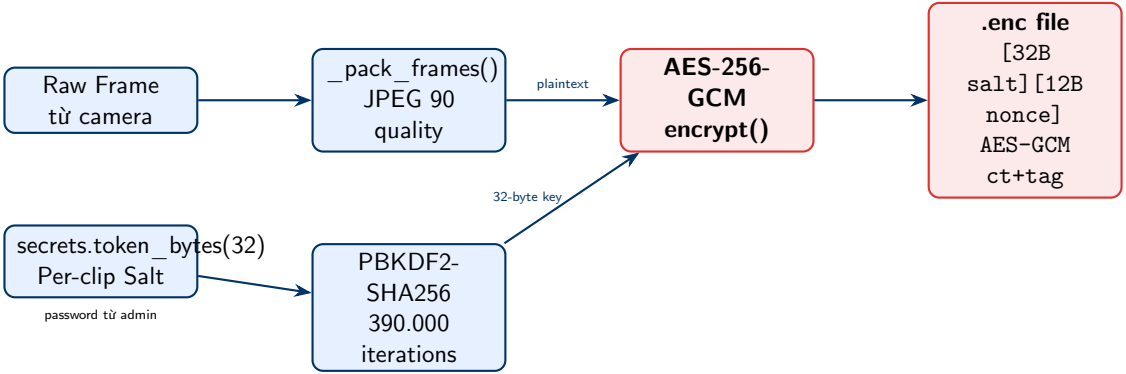
✔ Tại sao FrameReader thread riêng?

`cv2.VideoCapture.read()` blocking.
Thread riêng giúp frame luôn fresh,
không bị stale trong khi YOLO đang chạy.

✔ BBox Smoother

EMA: $\hat{b}_t = \alpha \cdot b_t + (1 - \alpha) \cdot \hat{b}_{t-1}$
$\alpha = 0.7$: responsive nhưng mượt.
Loại bỏ jitter bounding box frame-by-frame.

Security Architecture — Tổng Quan



Per-clip Salt

Mỗi clip = 1 salt 32B ngẫu nhiên.
Same password ⇒ **different**

AES-GCM

Authenticated Encryption.
Auth Tag 128-bit.
Sửa 1 bit ⇒ decrypt fail.

PBKDF2

390.000 rounds SHA256.
Brute-force 8-char:
≈ hàng trăm nghìn năm.

AES-256-GCM — Code Chi Tiết

Listing 1: src/clip_recorder.py

```python
def encrypt_clip(frames, fps, password):
    # Step 1: Serialize frames
    plaintext = _pack_frames(frames, fps)
    # [4B n_frames][4B fps*1000]
    # for each: [4B len][JPEG bytes]

    # Step 2: Random per-clip salt
    salt  = secrets.token_bytes(32)  # 256-bit
    nonce = secrets.token_bytes(12)  # 96-bit GCM

    # Step 3: Key derivation
```

Listing 2: src/clip_recorder.py

```python
def decrypt_clip(data, password):
    if len(data) < 44:
        raise ValueError("Corrupted")

    # Unpack file format
    salt       = data[:32]
    nonce      = data[32:44]
    ciphertext = data[44:]

    # Re-derive same key
    key = derive_clip_key(password, salt)

    try:
```

PBKDF2-SHA256 + Admin Authentication

Listing 3: src/admin_auth.py

```python
# OWASP 2023 minimum
_ITERATIONS = 390_000
_KEY_LEN = 32  # 256-bit

def _pbkdf2(password, salt,
            iterations=_ITERATIONS
                ):
    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=_KEY_LEN,
        salt=salt,
        iterations=iterations,
    )
    return kdf.derive(
        password.encode("utf-8"))
```

Credentials File

```
{
  "salt": "aGVsbG8h...(base64)
      ",
  "hash": "c2VjcmV0...(base64)
      ",
  "iterations": 390000,
  "created_at": 1741234567.89
}
```

Không có plaintext password.
Ngay cả dev cũng không biết password.

Timing Attack — Quan trọng!

a == b: dừng ở byte khác đầu tiên

Blur Engine — 7 Privacy Modes

Mode	Kỹ thuật	Mức độ
`blur`	Gaussian downscale	Nhẹ
`pixelate`	Bilinear+Nearest	Nhẹ-TB
`solid`	Constant fill	Nhanh
`headcloak`	Mix+Noise+GBlur	**Mạnh***
`neckup`	Expand+Cloak	Mạnh
`obliterate`	4-step pipeline	**Rất mạ**
`silhouette`	Extreme cloak	**Cực mạ**

*Default của hệ thống

Expand box trước khi blur

Không chỉ blur đúng mặt. `headcloak` expand:

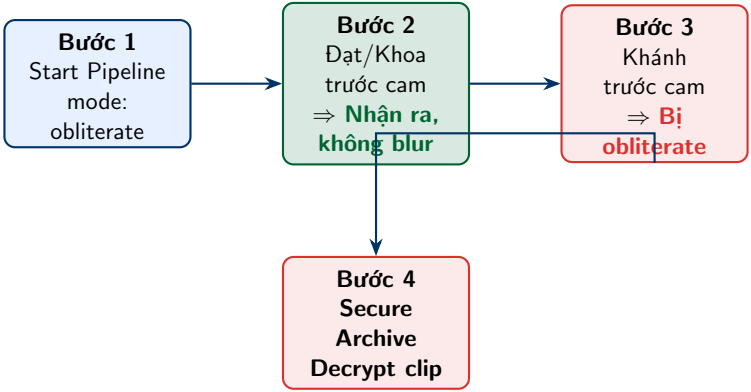
- Left/Right: $\pm 35\% - 70\%$ width

Listing 4: privacy.py — obliterate

```
def _obliterate_features(roi
    ,...):
  """4-step pipeline,
     IRREVERSIBLE"""

  # Step 1: Compress 8%
     resolution
  # Destroy all facial details
  w, h = roi.shape[:2]
  small = cv2.resize(roi,
    (int(w*0.08), int(h*0.08)),
    interpolation=INTER_AREA)
  work = cv2.resize(small, (w,
     h),
    interpolation=INTER_NEAREST
      )
```

Demo — End-to-End Flow



Kết quả mong đợi

- Đạt/Khoa: tên hiện, không blur

URLs Demo

Demo — So Sánh 5 Privacy Modes

blur	pixelate	headcloak	obliterate	silhouette
Gaussian mờ nhẹ	Mosaic 16px	Đầu+noise (default)	4-step pipeline	Extreme cloak

Chọn mode nào?

Use case	Mode
Xem tổng quan nhanh	`blur / pixelate`
Gia đình hàng ngày	`headcloak`
Bảo mật cao	`obliterate`
Pháp lý / nghiên cứu	`silhouette`

Performance

Mode	CPU	FPS*
blur	Thấp	∼25
pixelate	Rất thấp	∼28
headcloak	Trung bình	∼18
obliterate	Cao	∼12
silhouette	Rất cao	∼8

*CPU

mid-range, proc_width=640

Kết Quả Đạt Được

✓ Privacy

- 7 chế độ ẩn danh từ nhẹ đến cực mạnh
- Real-time inference ∼15–20 FPS
- Thành viên đã đăng ký: không bị che
- Người lạ: ẩn danh hoàn toàn
- Hot-reload mode không restart

✓ Usability

- RESTful API đầy đủ (FastAPI + OpenAPI)
- WebSocket + MJPEG streaming
- 4 preset sẵn sàng dùng
- Frontend HTML 6 trang, responsive
- Admin & User role separation

✓ Security

- AES-256-GCM: mã hóa + authenticated
- PBKDF2-SHA256 390.000 iter (OWASP 2023)
- Per-clip unique salt: không reuse key
- Constant-time comparison: no timing leak

✓ Recognition

- YOLOv11n-face: nano, fast, accurate
- InsightFace buffalo_l: 512-dim embedding
- Vectorized batch matching O(n)
- BBox EMA smoother: no jitter
- 2 thành viên đăng ký, 100 mẫu/người

Technology Stack

Lớp	Công nghệ	Phiên bản	Vai trò
Detection	Ultralytics YOLO	v11	Face bounding box
Recognition	InsightFace	buffalo_l	512-dim embedding
Image	OpenCV	$\geq$4.8	Frame processing
Backend	FastAPI	$\geq$0.115	REST + WebSocket
Server	Uvicorn	$\geq$0.30	ASGI server
Crypto	cryptography	$\geq$43.0	AES-GCM, PBKDF2
Numeric	NumPy	$\geq$1.24	Embedding vectors
Frontend	HTML + Tailwind	CSS v3	Neobrutalism UI
Language	Python	3.13	Backend runtime

Lệnh khởi động

```
uvicorn src.app_api:app --host 0.0.0.0 --port 8000 --reload
```

Câu Hỏi Thường Gặp

Q: AES-GCM vs AES-CBC?

GCM = **Authenticated Encryption**.
Auth Tag 128-bit bảo vệ integrity.
CBC chỉ encrypt, không verify.
Với bằng chứng pháp lý: integrity bắt buộc.

Q: 390.000 iterations?

OWASP 2023 minimum.
Đủ chậm để brute-force không khả thi.
Đủ nhanh để UX không bị ảnh hưởng
(encrypt/decrypt <1 lần/phút).

Q: Quên password?

Không recover được — **by design**.
Zero Knowledge of Plaintext.
Production: Key Escrow hoặc
Shamir Secret Sharing.

Q: False positive nhận diện?

Người lạ nhầm thành member → privacy leak.
Mitigated: threshold 0.35 + 100 mẫu/người.
Production: threshold calibration + A/B test.

Q: GDPR compliance?

Privacy by Design (Art.25): encrypt by default, limited access, right to delete (DELETE /members).

Cảm ơn Thầy/Cô đã lắng nghe!

XLA Face Privacy System

Privacy by Default · Evidence on Demand

Nhóm thực hiện

Đạt	Leader	AES-256-GCM · PBKDF2 · Blur
Khánh	Backend	FastAPI · Pipeline · Stream
Khoa	Detection	YOLOv11 · InsightFace · DB

Repository

`Poicitaco/XLA_T3`

Branch: `feature/cipher`

Port 8000

Sẵn sàng trả lời câu hỏi!